САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Кащеев Максим Николаевич
Группа:
J3211

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача:

Нужно привязать то, в соответствии с темой в ЛАБ1 к внешнему API средствами fetch/axios/xhr. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию.

Например, для приложения для просмотра прогнозов погоды задание выглядит следующим образом:

Реализовать получение погоды (прогноз на ближайшие 7 дней) из открытого API OpenWeatherMap, взависимости от геолокации пользователя. Реализовать вывод полученного прогноза в виде 7 карточек в три ряда (первый ряд - крупная карточка, второй ряд - три карточки в меньшем размере, третий ряд - четыре карточки в маленьком размере).

Ход работы:

Реализация мокового сервера (server.js):

Сервер написан на Node.js с использованием фреймворка Express. В качестве хранилища данных используется файл db.json, который содержит массивы users, models, datasets, spaces.

Реализованные эндпоинты:
POST /auth/register: регистрация нового пользователя
POST /auth/login: вход, выдача JWT-токена
GET /me: Данные текущего пользователя
GET /api/models: Список моделей (фильтр: search, category)Нет
GET /api/model/:id: Одна модель по id
GET /api/datasets: Список датасетов (фильтр: search, category)
GET /api/dataset/:id: Один датасет по id
GET /api/spaces: Список пространств (фильтр: search)
GET /api/space/:id: Одно пространство по id

Авторизация реализована через JWT (JSON Web Token).

Реализация клиентского API-модуля (api.js)

Создан единый модуль api.js, экспортирующий все функции для работы с сервером. Все запросы реализованы через браузерный Fetch API (async/await):

getModels(filters): получение списка моделей с фильтрами

getDatasets(filters): получение датасетов

getSpaces(filters): получение пространств

getModelById(id), getDatasetById(id), getSpaceById(id): получение объекта по ID

isLoggedIn(): проверка наличия токена в localStorage

logout(): - очистка localStorage и редирект

Токен авторизации сохраняется в localStorage при входе и отправляется в заголовке Authorization: Bearer <token> при защищённых запросах.

Подключение к API на страницах:

Главная страница (index.html / index-main.js):

При загрузке вызывается getModels(), затем данные рендерятся в карусель.

Поиск и теги-фильтры передают параметры в getModels

Проверка авторизации (isLoggedIn) управляет отображением navbar.

Каталог моделей (models.html):

GET /api/models?category=… - фильтрация по категориям (CV, NLP, Tabular и др.).

GET /api/models?search=… - текстовый поиск по полям title, description, tagLabel.

Клик по карточке - переход на model-detail.html?id=<id>.

Детальная страница (model-detail.html):

ID модели считывается из URLSearchParams.

Запрос GET /api/model/:id: рендер данных конкретной модели.

Датасеты и пространства (datasets.html, spaces.html):

Аналогичная схема: GET /api/datasets, GET /api/spaces с фильтрами.

Лайки и избранное хранятся в localStorage (с проверкой авторизации перед действием).

Личный кабинет (account.html):

Проверяет наличие токена; при отсутствии идёт перенаправление на registration.html.

Запрос GET /me с заголовком Authorization: Bearer <token> для верификации сессии.

Избранные модели згружаются по ID из localStorage через GET /api/model/:id.

Регистрация и вход (registration.html, login.html):

POST /auth/register и POST /auth/login через fetch с телом { name, email, password }.

Успешный ответ сохраняет { token, user } в localStorage, после чего происходит перенаправление.

Структура данных (db.json):

users - зарегистрированные пользователи (id, name, email, passwordHash)

models - модели ML (id, title, category, tagLabel, description, downloads, likes, slug и др.)

datasets - наборы данных (id, title, category, description, extra1, downloads, likes и др.)

spaces - демо-пространства (id, title, description, badge, runs, likes, slug)

Вывод

В ходе лабораторной работы был реализован полноценный мок-бэкенд на Node.js/Express, имитирующий работу реального REST API. Фронтенд платформы MLDrop полностью переведён на динамическую загрузку данных через браузерный Fetch API. Реализована авторизация с выдачей JWT-токена, хэшированием паролей через bcrypt и проверкой токена на защищённых маршрутах. Данные для всех страниц (модели, датасеты, пространства) поступают исключительно из API. Таким образом, проект соответствует требованиям задания: моковое API на JSON-сервере + авторизация + взаимодействие через fetch.